

JOBSHEET I

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Konsep Pemrograman Berorientasi Objek

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 28 Agustus 2026

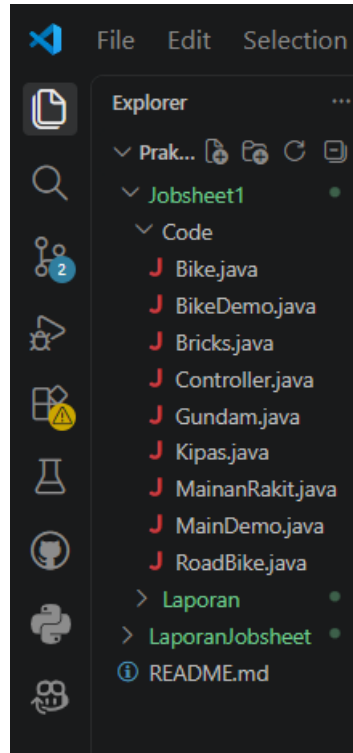
POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141

TAHUN 2026-2027

2. Struktur Project



Gambar 2. 1 Struktur Project

Penjelasan: Gambar di atas menunjukkan struktur direktori project di dalam folder Jobsheet1. Folder utama Code berisi seluruh file *source code* Java yang terbagi menjadi dua bagian, yaitu file percobaan (Bike.java sebagai class induk, RoadBike.java sebagai class turunan, dan BikeDemo.java sebagai class utama/main) serta file tugas praktikum (MainanRakit.java sebagai induk, Gundam.java dan Bricks.java sebagai turunan, Kipas.java dan Controller.java sebagai class mandiri, serta MainDemo.java sebagai class utama untuk menjalankan tugas).

3. Percobaan

3.1 Percobaan 1

1. Buka Netbeans atau menggunakan editor java, buat project BikeDemo.
2. Buat class Bike. Klik kanan pada package BikeDemo – New – Java Class.
3. Ketikkan kode class Bike dibawah ini.

```

1  package Jobsheet1.BikeDemo;
2
3  public class Bike {
4      private String brand;
5      private int speed;
6      private int gear=1;
7      // Gear 1: max 5 km/h, gear 2: max 10 km/h, ... gear 6: max 60 km/h
8      private final int[] gearSpeedLimits = { 5, 10, 25, 30, 40, 60 };
9
10     public void setBrand(String brandName) {
11         this.brand = brandName;
12     }
13
14     public void gearChanges(int gearValue) {
15         if (gearValue < 1 || gearValue > 6) {
16             System.out.println("Invalid gear value. Please select a gear between 1 and 6.");
17         } else {
18             gear = gearValue;
19         }
20     }
21
22     public int speedAcceleration(int increment) {
23         speed += increment;
24         if (speed > gearSpeedLimits[gear - 1]) {
25             speed = gearSpeedLimits[gear - 1];
26         }
27         return speed;
28     }
29
30     public int speedDeceleration(int decrement) {
31         speed -= decrement;
32         if (speed < 0) {
33             speed = 0;
34         }
35         return speed;
36     }
37
38     public void printInfo() {
39         System.out.println("Bike Brand: " + brand);
40         System.out.println("Current Speed: " + speed + " km/h");
41         System.out.println("Current Gear: " + gear);
42         System.out.println("");
43     }
44 }
45

```

Gambar 3. 1 Class Bike Code

Penjelasan: Pembuatan class Bike sebagai cetakan dasar objek sepeda. Di dalam class ini dibuat atribut seperti merek (brand), kecepatan (speed), dan posisi gigi (gear), lengkap dengan method untuk mengatur merek, mengganti gigi, menambah/mengurangi kecepatan dengan batas maksimal tertentu, serta method untuk menampilkan informasinya ke layar.

4. Kemudian pada class main, ketikkan kode berikut ini.

```

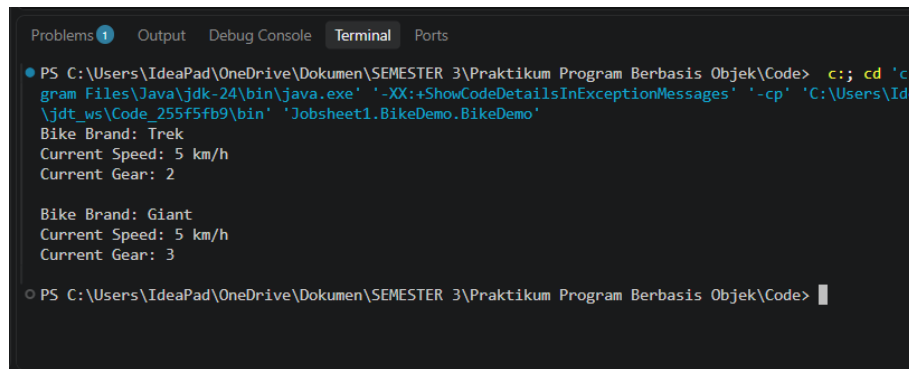
1  package Jobsheet1.BikeDemo;
2
3  public class BikeDemo {
4      public static void main(String[] args){
5          Bike mountainBike1 = new Bike();
6          Bike mountainBike2 = new Bike();
7          RoadBike roadBike1 = new RoadBike();
8
9          mountainBike1.setBrand("Trek");
10         mountainBike1.speedAcceleration(10);
11         mountainBike1.gearChanges(2);
12         mountainBike1.printInfo();
13
14         mountainBike2.setBrand("Giant");
15         mountainBike2.speedAcceleration(20);
16         mountainBike2.gearChanges(3);
17         mountainBike2.printInfo();
18     }
19 }

```

Gambar 3. 2 Class Main Code

Penjelasan: Menampilkan kode pada method main di class BikeDemo yang bertugas menjalankan program. Di sini kita membuat dua objek nyata dari cetakan Bike, yaitu mountainBike1 dan mountainBike2, lalu mengatur mereka masing-masing, menambah kecepatannya, mengubah giginya, dan memanggil fungsi printInfo() untuk mencetak datanya.

5. Cocokkan hasilnya:



```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Code> .c; cd 'c:\gram Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Code\Jobsheet1.BikeDemo.BikeDemo'
```

Bike Brand: Trek
Current Speed: 5 km/h
Current Gear: 2

Bike Brand: Giant
Current Speed: 5 km/h
Current Gear: 3

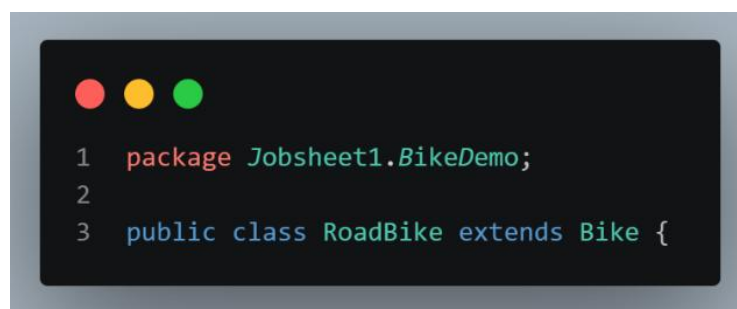
```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Code> |
```

Gambar 3. 3 Percobaan 1 Hasil

Penjelasan: Hasil keluaran (*output*) di terminal saat program BikeDemo pertama kali dijalankan. Terlihat data sepeda merek *Trek* dan *Giant* berhasil tampil dengan benar sesuai nilai gigi dan kecepatan yang telah dibatasi oleh aturan pada class

3.2 Percobaan 2

1. Masih pada project BikeDemo. Buat class RoadBike.
2. Tambahkan kode extends Bike pada deklarasi class RoadBike. Kode extends ini menandakan bahwa class RoadBike mewarisi class Bike.



```
1 package Jobsheet1.BikeDemo;  
2  
3 public class RoadBike extends Bike {
```

Gambar 3. 4 Extends

Penjelasan: Pembuatan class baru bernama RoadBike dengan menambahkan kata kunci extends Bike. Kode ini menerapkan konsep *inheritance* (pewarisan), yang artinya class RoadBike dijadikan sebagai class anak yang otomatis mewarisi semua atribut dan method milik class induknya (Bike).

3. Kemudian lengkapi kode RoadBike seperti berikut ini:

A screenshot of a code editor showing the implementation of the RoadBike class. The code is written in Java and includes package declarations, class inheritance, a private attribute, a setter method, and an overridden printInfo method. The code is as follows:

```
1 package Jobsheet1.BikeDemo;
2
3 public class RoadBike extends Bike {
4     private int tireWidth;
5
6     public void setTireWidth(int width) {
7         this.tireWidth = width;
8     }
9
10    @Override
11    public void printInfo() {
12        super.printInfo();
13        System.out.println("Tire Width: " + tireWidth + " mm");
14        System.out.println("bike Type: Road Bike");
15    }
16
17 }
18
19 }
```

Gambar 3. 5 RoadBike Code

Penjelasan: Isi lengkap dari class RoadBike yang sudah ditambahkan atribut khusus ukuran lebar ban (tireWidth). Selain itu, terdapat method printInfo() yang di-override dan menggunakan super.printInfo() agar data bawaan dari class Bike tetap tercetak bersamaan dengan data tipe sepeda dan lebar bannya.

4. Kemudian pada class main, tambahkan kode berikut ini:

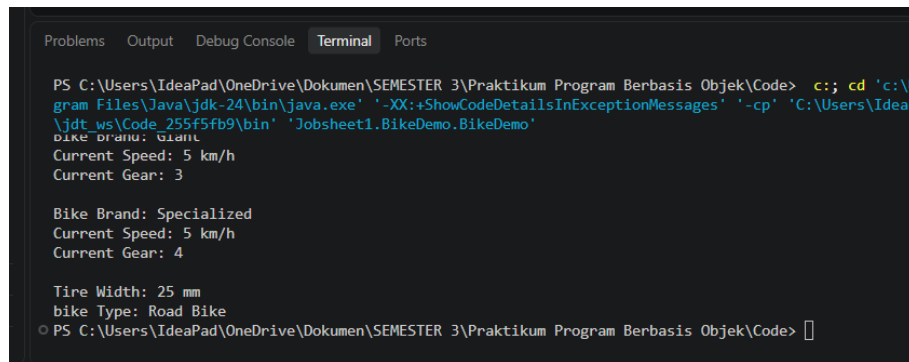
A screenshot of a code editor showing the implementation of the BikeDemo class. The code is written in Java and includes package declarations, class inheritance, a static main method, and various method calls to create and manipulate Bike objects. The code is as follows:

```
1 package Jobsheet1.BikeDemo;
2
3 public class BikeDemo {
4     public static void main(String[] args){
5         Bike mountainBike1 = new Bike();
6         Bike mountainBike2 = new Bike();
7         RoadBike RoadBike1 = new RoadBike();
8
9         mountainBike1.setBrand("Trek");
10        mountainBike1.speedAcceleration(10);
11        mountainBike1.gearChanges(2);
12        mountainBike1.printInfo();
13
14        mountainBike2.setBrand("Giant");
15        mountainBike2.speedAcceleration(20);
16        mountainBike2.gearChanges(3);
17        mountainBike2.printInfo();
18
19        RoadBike1.setBrand("Specialized");
20        RoadBike1.setTireWidth(25);
21        RoadBike1.speedAcceleration(15);
22        RoadBike1.gearChanges(4);
23        RoadBike1.printInfo();
24    }
25 }
26 }
```

Gambar 3. 6 Class Main Code

Penjelasan: Memperlihatkan pembaruan pada class BikeDemo, di mana kita menambahkan pembuatan satu objek baru bernama RoadBike1. Objek ini kemudian diisi merknya, diatur lebar bannya menjadi 25 mm, dinaikkan kecepatannya, diubah giginya, dan dipanggil method printInfo() untuk menampilkan informasi lengkapnya.

5. Cocokkan hasilnya:



```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Code> c:: cd 'C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Code' & java -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Code\bin' 'Jobsheet1.BikeDemo.BikeDemo'
Bike Brand: Giant
Current Speed: 5 km/h
Current Gear: 3

Bike Brand: Specialized
Current Speed: 5 km/h
Current Gear: 4

Tire Width: 25 mm
bike Type: Road Bike
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Code>
```

Gambar 3. 7 Hasil Percobaan 2

Penjelasan: Hasil akhir eksekusi program di terminal setelah class turunan ditambahkan. Terlihat objek RoadBike berhasil menampilkan informasi standar sepeda sekaligus informasi tambahannya, yaitu ukuran lebar ban (*Tire Width: 25 mm*) dan tipe sepeda (*Road Bike*).

4. Kesimpulan

Pada class **Bike.java**, seperti yang telah dijelaskan oleh Pak Subhi bahwasanya class ini berfungsi sebagai cetakan utama. Di dalam cetakan ini, kita menyimpan *state* (ciri-ciri) dari objek sepeda, seperti brand, speed, dan gear. Selain itu, terdapat juga *behavior* (aksi/fungsi) yang bisa dilakukan oleh sepeda, seperti menambah kecepatan (*speedAcceleration*), mengganti gigi (*gearChanges*), dan mengurangi kecepatan (*speedDeceleration*).

Pada class **RoadBike.java**, class ini bertindak sebagai anak atau turunan langsung dari class Bike menggunakan extends. Karena dia anak dari Bike, dia otomatis mewarisi semua sifat dari induknya, namun punya ciri khas tersendiri yang tidak dimiliki sepeda biasa, yaitu ukuran lebar ban (*tireWidth*).

Sedangkan pada class **BikeDemo.java**, file ini berfungsi sebagai tempat uji coba atau tempat kita membuat objek nyata dari cetakan **Bike.java** dan **RoadBike.java**. Di sinilah kita memberi merk, mengatur gigi, menambah kecepatan, dan menampilkan informasinya ke layar.

5. Pertanyaan

1. Jelaskan perbedaan antara object dengan class!

2. Jelaskan alasan gear dan brand dapat menjadi atribut dari object Bike!
3. Sebutkan salah satu kelebihan utama dari pemrograman berorientasi objek dibandingkan dengan pemrograman prosedural!
4. Apakah diperbolehkan melakukan pendefinisian dua buah atribut dalam satu baris kode seperti “public String nama, alamat;”?
5. Pada class RoadBike, jelaskan alasan atribut brand, speed, dan gear tidak lagi ditulis di dalam class tersebut!

Jawab:

1. Class: Gambar denah atau cetakannya. Ini baru sebatas rancangan di atas kertas yang menentukan nanti objek punya *state* apa dan *behavior* apa. Object: Barang aslinya yang sudah jadi dan nyata dibuat berdasarkan cetakan tersebut. **Contoh:** jika classnya bike, objectnya bisa mountain bike, road bike, dan macam macam jenis bike lainnya.
2. Jika kita berpacu pada dunia nyata, setiap sepeda pasti punya identitas fisik atau ciri khas tersebut. Sepeda pasti punya merek (brand) dan gigi yang sedang dipakai (gear). Karena keduanya merupakan ciri-ciri (*state*) yang melekat pada sepeda, maka di dalam program dijadikan sebagai atribut.
3. kode jadi lebih rapi dan bisa dipakai berulang kali (*reusable*). Kita tidak perlu menulis kode yang sama dari nol setiap kali ingin membuat variasi baru. Cukup bikin satu cetakan induk (Bike), lalu kalau butuh variasi lain tinggal kita turunkan ke anak (RoadBike), dan lebih mudah di pahami oleh orang lain karena strukturnya jelas.
4. Dari yang saya pelajari pada mata kuliah sebelumnya seperti daspro, hal tersebut sah dan boleh tidak akan menghasilkan error, namun jika kita menggunakan oop menurut saya lebih mudah di pahami dan di baca jika satu persatu, apalagi jika didefinisikan lebih dari 2, walaupun tidak akan terjadi error namun kode akan sangat panjang ke samping sehingga di khawatirkan beberapa atribut tidak terbaca atau terlewat.
5. Karena class **RoadBike.java** posisinya sebagai anak yang sudah mewarisi semua isi dari class **Bike.java** lewat perintah extends Bike. Jadi, atribut brand, speed, dan gear otomatis sudah ikut terbawa dari induknya tanpa perlu kita ketik ulang di class **RoadBike.java**.

6. Tugas Praktikum

1. Lakukan langkah-langkah berikut supaya tugas praktikum yang dikerjakan tersistematis:
 - a. Foto 4 buah objek di sekitar kalian dengan 2 objek di antaranya merupakan objek yang mengandung konsep pewarisan (inheritance), contoh: kulkas, kursi, meja ruang tamu, meja belajar sehingga diketahui meja ruang tamu dan meja belajar mewarisi objek meja!



Gambar 6. 1 Kipas Angin



Gambar 6. 2 Controller Game



Gambar 6. 3 Mainan Gundam



Gambar 6. 4 Mainan Bricks

Keterangan: Berikut ini adalah foto dari 4 objek yang ada di sekitarku, mulai dari Gambar 6.1 sampai Gambar 6.4. Pada Gambar 6.3 dan Gambar 6.4, kedua objek tersebut merupakan turunan yang mewarisi sifat dan ciri-ciri dari objek MainanRakit.

b. Lakukan pengamatan terhadap 4 objek tersebut untuk menentukan atribut dan methodnya!

1. Kipas Angin (Kipas)

- **Atribut:** brand (merek kipas) dan speed (level kecepatan kipas).
- **Method:**
 - tambahSpeed(): Menaikkan level kecepatan kipas.
 - matikanKipas(): Mematikan kipas (speed diatur ke 0).
 - cetakInfo(): Menampilkan merek dan level speed saat ini.

2. Gamepad Controller (Controller)

- **Atribut:** brand (merek controller) dan tombolX.
- **Method:**
 - tekanTombolX(): Melakukan aksi menekan tombol X.
 - lepasTombolX(): Melepas tombol X ke kondisi semula.
 - cetakInfo(): Menampilkan merek dan status tombolX.

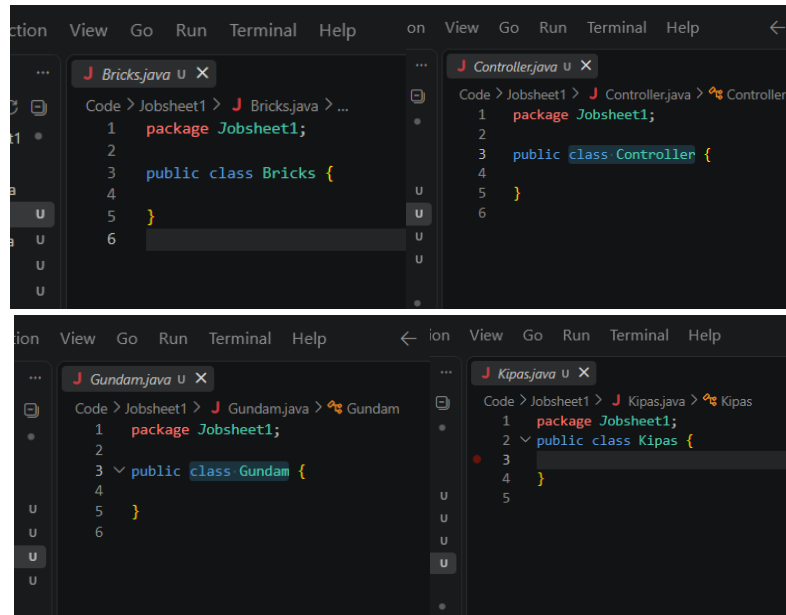
3. Gundam (Gundam mewarisi MainanRakit)

- **Atribut:** tier (tingkatan kit rakitan, misal: "HG", "RG") dan senjata.
- **Method:**
 - pasangSenjata(): Memasang senjata pada figur Gundam.
 - lepasSenjata(): Melepas senjata dari tangan Gundam.
 - cetakInfo(): Menampilkan info lengkap Gundam (nama, harga, tier, dan senjata).

4. Bricks (Bricks mewarisi MainanRakit)

- **Atribut:** seri (tema seri bricks) dan jumlahPcs (total kepingan balok).
- **Method:**
 - rakit(): Merakit kepingan balok bricks.
 - bongkar(): Membongkar kepingan bricks yang sudah tersusun.
 - cetakInfo(): Menampilkan info lengkap Bricks (nama, harga, seri, dan jumlahPcs).

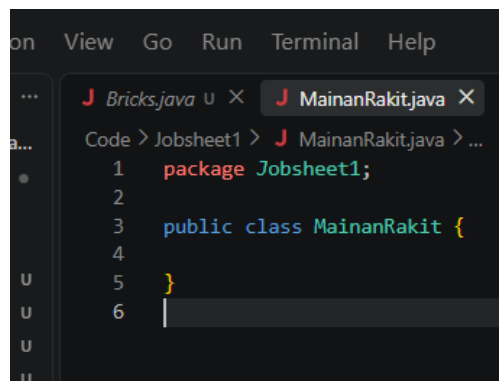
c. Berdasarkan 4 buah objek tersebut, buat class nya dalam Bahasa pemrograman Java!



Gambar 6. 5 Class 4 object

Keterangan: Menunjukkan pembuatan 4 file class dasar dalam package Jobsheet1, yaitu Kipas.java, Controller.java, Gundam.java, dan Bricks.java sebagai cetakan awal dari objek-objek yang telah diamati.

- d. Perlu diperhatikan bahwa terdapat dua class hasil pewarisan sehingga perlu menambah satu class baru sebagai class yang mewarisi dua class tersebut!



Gambar 6. 6 Class MainanRakit.java (induk)

Keterangan: Menunjukkan penambahan class baru bernama MainanRakit.java, Dimana class ini merupakan class yang akan mewarisi Gundam dan Bricks (class induk).

e. Tambahkan dua atribut untuk setiap class!



Gambar 6. 7 Class Atribut

Keterangan: Menunjukkan penambahan 2 buah atribut privat pada setiap class. Pada class Gundam dan Bricks, telah ditambahkan kata kunci extends MainanRakit untuk menghubungkan relasi pewarisan dengan class induknya (MainanRakit.java).

f. Tambahkan tiga method untuk setiap class termasuk method cetak informasi!

```
1 package Jobsheet1;
2
3 public class MainanRakit {
4     private String nama;
5     private int harga;
6
7     public void setName(String nama) {
8         this.nama = nama;
9     }
10
11    public void setHarga(int harga) {
12        this.harga = harga;
13    }
14
15    // Method 1
16    public void mulaiRakit() {
17        System.out.println("Memulai proses perakitan " + nama + "...");
18    }
19
20    // Method 2
21    public void beriDiskon(int persen) {
22        int potongan = (this.harga * persen) / 100;
23        this.harga -= potongan;
24        System.out.println(nama + ": Diberi diskon " + persen + "%, harga jadi Rp " + this.harga);
25    }
26
27    // Method 3
28    public void cetakInfo() {
29        System.out.println("Nama: " + nama);
30        System.out.println("Harga: Rp " + harga);
31    }
32 }
```

```
1 package Jobsheet1;
2
3 public class Controller {
4     private String brand;
5     private String tombolX = "Tidak Ditekan";
6
7     public void setBrand(String brand) {
8         this.brand = brand;
9     }
10
11    // Method 1
12    public void tekanTombolX() {
13        tombolX = "Ditekan";
14        System.out.println(brand + ": Tombol X ditekan.");
15    }
16
17    // Method 2
18    public void lepasTombolX() {
19        tombolX = "Tidak Ditekan";
20        System.out.println(brand + ": Tombol X dilepas.");
21    }
22
23    // Method 3
24    public void cetakInfo() {
25        System.out.println("=== INFO CONTROLLER ===");
26        System.out.println("Brand: " + brand);
27        System.out.println("Status Tombol X: " + tombolX);
28        System.out.println("-----");
29    }
30 }
```

```
1 package Jobsheet1;
2
3 public class Kipas {
4     private String brand;
5     private int speed;
6
7     public void setBrand(String brand) {
8         this.brand = brand;
9     }
10
11    // Method 1
12    public void tambahSpeed(int tambah) {
13        speed += tambah;
14        if (speed > 3) {
15            speed = 3;
16        }
17        System.out.println(brand + ": Kecepatan naik ke level " + speed);
18    }
19
20    // Method 2
21    public void matikanKipas() {
22        speed = 0;
23        System.out.println(brand + ": Kipas dimatikan.");
24    }
25
26    // Method 3
27    public void cetakInfo() {
28        System.out.println("=== INFO KIPAS ===");
29        System.out.println("Brand: " + brand);
30        System.out.println("Speed: Level " + speed);
31        System.out.println("-----");
32    }
33 }
```

```

1 package Jobsheet1;
2
3 public class Gundam extends MainanRakit {
4     private String tier;
5     private String senjata;
6
7     public void setTier(String tier) {
8         this.tier = tier;
9     }
10
11     // Method 1
12     public void pasangSenjata(String namaSenjata) {
13         this.senjata = namaSenjata;
14         System.out.println("Senjata " + namaSenjata + " berhasil dipasang.");
15     }
16
17     // Method 2
18     public void lepasSenjata() {
19         System.out.println("Senjata " + this.senjata + " telah dilepas.");
20         this.senjata = "Tangan Kosong";
21     }
22
23     // Method 3
24     @Override // Mengganti atau Menimpa aturan bawaan dari Induk.
25     public void cetakInfo() {
26         System.out.println("=== INFO GUNDAM ===");
27         super.cetakInfo();
28         System.out.println("Tier: " + tier);
29         System.out.println("Senjata: " + (senjata != null ? senjata : "Belum ada senjata"));
30         System.out.println("-----");
31     }
32 }

```

```

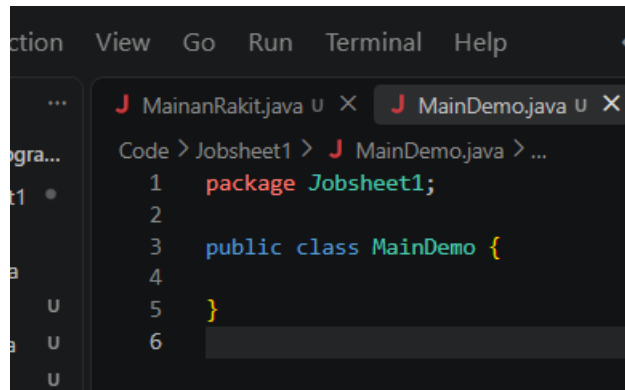
1 package Jobsheet1;
2
3 public class Bricks extends MainanRakit {
4     private String seri;
5     private int jumlahPcs;
6
7     public void setSeri(String seri) {
8         this.seri = seri;
9     }
10
11     public void setJumlahPcs(int jumlahPcs) {
12         this.jumlahPcs = jumlahPcs;
13     }
14
15     // Method 1
16     public void rakit(int dipasang) {
17         System.out.println("Sedang merakit " + dipasang + " keping bricks seri " + seri + ".");
18     }
19
20     // Method 2
21     public void bongkar() {
22         System.out.println("Bricks seri " + seri + " telah dibongkar.");
23     }
24
25     // Method 3
26     @Override
27     public void cetakInfo() {
28         System.out.println("=== INFO BRICKS ===");
29         super.cetakInfo();
30         System.out.println("Seri: " + seri);
31         System.out.println("Jumlah Pcs: " + jumlahPcs + " pcs");
32         System.out.println("-----");
33     }
34 }

```

Gambar 6. 8 Method

Keterangan: Menambahkan 3 buah method untuk tiap class termasuk method cetakInfo(). Pada class turunan (Gundam dan Bricks), dipakai @Override dan super.cetakInfo() agar info dari induknya tetap ikut tercetak bersama atribut khususnya.

- g. Tambahkan satu class Demo sebagai main!



Gambar 6. 9 DemoMain class

Keterangan: Menunjukkan pembuatan class MainDemo

- h. Instansiasikan satu buah objek untuk setiap class!



Gambar 6. 10 Instansiasi

Keterangan: Menunjukkan proses instansiasi (pembuatan objek nyata di dalam memori) menggunakan kata kunci new untuk kelima class yang ada, yaitu objek kipasKamar, controllerPS, gundamWing, brickRusa, dan mainanRakit1.

- i. Terapkan setiap method untuk setiap objek yang dibuat!



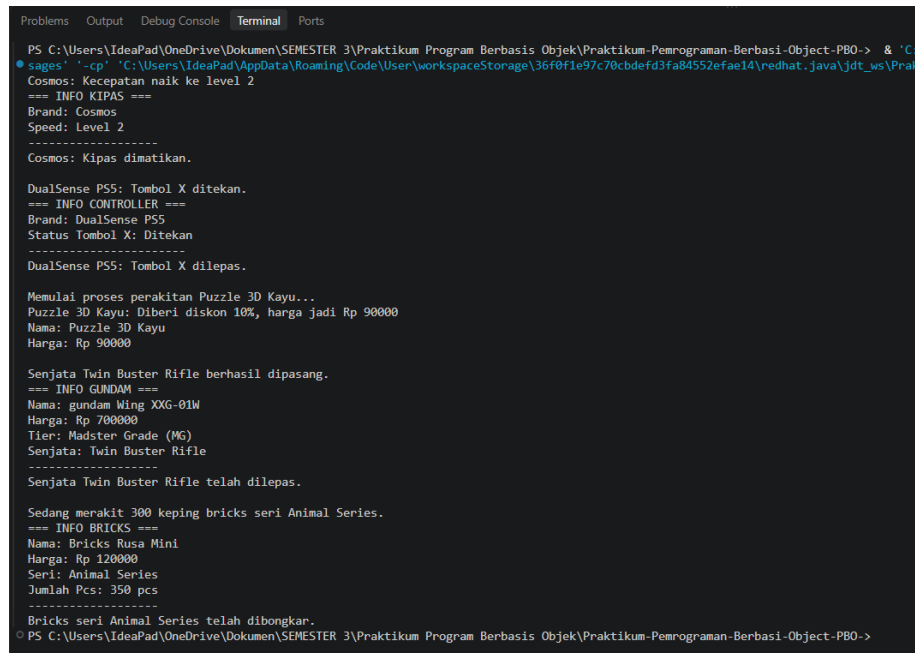
```
1 package Jobsheet1;
2
3 public class MainDemo {
4     public static void main(String[] args) {
5
6         // Objek Kipas
7         Kipas kipasKamar = new Kipas();
8         kipasKamar.setBrand("Cosmos");
9         kipasKamar.tambahSpeed(2);
10        kipasKamar.cetakInfo();
11        kipasKamar.matikanKipas();
12
13        System.out.println();
14
15        // Objek Controller
16        ControllerPS controllerPS = new ControllerPS();
17        controllerPS.setBrand("DualSense PSS");
18        controllerPS.tekanTombolX();
19        controllerPS.cetakInfo();
20        controllerPS.lepasTombolX();
21
22        System.out.println();
23
24        // Objek Mainan Rakit
25        MainanRakit mainanRakit1 = new MainanRakit();
26        mainanRakit1.setNama("Puzzle 3D Kayu");
27        mainanRakit1.setHarga(100000);
28        mainanRakit1.mulaiRakit();
29        mainanRakit1.beriDiskon(10);
30        mainanRakit1.cetakInfo();
31
32        System.out.println();
33
34        // Objek Gundam
35        Gundam gundamWing = new Gundam();
36        gundamWing.setNama("gundam Wing XXG-01W");
37        gundamWing.setHarga(700000);
38        gundamWing.setTier("Madster Grade (MG)");
39        gundamWing.pasangSenjata("Twin Buster Rifle");
40        gundamWing.cetakInfo();
41        gundamWing.lepasSenjata();
42
43        System.out.println();
44
45        // Objek Bricks
46        Bricks brickRusa = new Bricks();
47        brickRusa.setNama("Bricks Rusa Mini");
48        brickRusa.setHarga(120000);
49        brickRusa.setSeri("Animal Series");
50        brickRusa.setJumlahPcs(350);
51        brickRusa.rakit(300);
52        brickRusa.cetakInfo();
53        brickRusa.bongkar();
54    }
55 }
```

Gambar 6. 11 Method

Keterangan: Menunjukkan penerapan lengkap seluruh method dan pengisian nilai atribut pada setiap objek yang telah diinstansiasi di dalam method main.

- j. Contoh yang telah disebutkan pada poin 1.a tidak diperbolehkan dipakai dalam pengerjaan tugas praktikum ini!

Hasil

A screenshot of a Java IDE's terminal window. The terminal shows the output of a Java program. The output is as follows:

```
PS C:\Users\IdeaPad\OneDrive\Drive\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> & "C:\sages" "-cp" "C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\36f0f1e97c70cbdefd3fa84552efae14\redhat.java\jdt_ws\Prak
Cosmos: Kecepatan naik ke level 2
=== INFO KIPAS ===
Brand: Cosmos
Speed: Level 2
-----
Cosmos: Kipas dimatikan.

DualSense PS5: Tombol X ditekan.
=== INFO CONTROLLER ===
Brand: DualSense PS5
Status Tombol X: Ditekan
-----
DualSense PS5: Tombol X dilepas.

Memulai proses perakitan Puzzle 3D Kayu...
Puzzle 3D Kayu: Diberi diskon 10%, harga jadi Rp 90000
Nama: Puzzle 3D Kayu
Harga: Rp 90000

Senjata Twin Buster Rifle berhasil dipasang.
=== INFO GUNDAM ===
Nama: gundam Wing XG-01W
Harga: Rp 700000
Tier: Master Grade (MG)
Senjata: Twin Buster Rifle
-----
Senjata Twin Buster Rifle telah dilepas.

Sedang merakit 300 keping bricks seri Animal Series.
=== INFO BRICKS ===
Nama: Bricks Rusa Mini
Harga: Rp 120000
Seri: Animal Series
Jumlah Pcs: 350 pcs
-----
Bricks seri Animal Series telah dibongkar.
PS C:\Users\IdeaPad\OneDrive\Drive\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO->
```

Gambar 6. 12 Hasil

Pembahasan: Hasil akhir program pada Gambar 6.12 menunjukkan bahwa seluruh method dari objek Kipas, Controller, MainanRakit, Gundam, dan Bricks berhasil dieksekusi dengan lancar tanpa error di terminal. Output ini secara langsung mencerminkan konsep dasar PBO yang dipelajari minggu ini: perancangan Class sebagai cetakan dasar dan Object sebagai wujud nyatanya di memori, pemanfaatan Attribute (State) serta Method (Behavior) untuk memanipulasi data objek (seperti menaikkan kecepatan kipas, menekan tombol controller, atau memberi diskon harga), serta penerapan Inheritance (Pewarisan) pada class Gundam dan Bricks. Pada terminal terlihat jelas bahwa Gundam dan Bricks sukses mewarisi data umum dari induknya (MainanRakit) berupa nama dan harga, lalu menggabungkannya secara rapi dengan atribut khusus masing-masing lewat penggunaan `@Override` dan `super.cetakInfo()`.

Kesimpulan Keseluruhan

Dari praktikum Jobsheet 1 ini dapat disimpulkan bahwa Pemrograman Berorientasi Objek (PBO) sangat memudahkan dalam memodelkan benda-benda di dunia nyata ke dalam bentuk program melalui perancangan *Class* dan pembuatan *Object*. Selain itu, fitur pewarisan sifat (*Inheritance*) terbukti membuat struktur kode menjadi jauh lebih rapi, teratur, dan hemat waktu (*reusable*), karena atribut maupun method umum pada class induk dapat langsung digunakan oleh class anak tanpa perlu ditulis ulang dari awal.

Lampiran

[Link Github Ahmad Rafid Riqkullah](#) Atau

**[https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-
/tree/main/Jobsheet1](https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet1)**